

RETAILMART V3 • SQL

Onboarding & First Queries

WhatsApp Announcement

ONBOARDING & YOUR FIRST SQL QUERIES

SQL Masterclass — Session 2

Today's Focus:

- Writing your FIRST SQL queries!
- Exploring a real database with psql commands
- Loading the REAL RetailMart database
- Touring 16 schemas and 55 tables of real data!

Why This Matters:

Every analyst starts the same way: connect to a real database, look around, and run that first query. Today you do exactly that — on 2.7 million rows of real retail data.

 Pro Tip: SELECT is the command you will use most in your career. Today it becomes muscle memory — next session we start BUILDING with DDL.

 GitHub Repo: <https://github.com/starlordali4444/postgresql>

Let's build something REAL today! See you all ON TIME! ⚡

— Siraj Sir

#PostgreSQL #SQLBasics #RetailMart



YOUR SQL JOURNEY SO FAR

Introduction to SQL & Installation

- What is SQL? The universal language for relational databases
- SQL 5 categories: DDL, DML, DQL, DCL, TCL
- Database vs Schema: Building vs Departments inside
- Installed PostgreSQL 18, pgAdmin 4, VS Code with SQLTools
- Client-Server architecture: pgAdmin (Client) talks to PostgreSQL (Server) on Port 5432



Today (Onboarding & First Queries)

- We're ready to load the real RetailMart database and write our FIRST real queries!
- Next session: DDL — CREATE, DROP and ALTER to build your own tables

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: SQL Basics & SELECT	35 mins	Writing basic SQL queries, SELECT as a calculator, System info queries
Part 2: Exploring a Real Database	35 mins	psql catalog commands: \l \c \dn \dt \d, Schema-qualified names, SELECT ... LIMIT on real tables
Part 3: Load the Real RetailMart	40 mins	Get the course repo, One-command import of 16 schemas / 55 tables, Verify the load worked
Part 4: Tour & Q&A	30 mins	Walk the 16 schemas, First queries on real data, What comes next: DDL

MAIN STORY: The School That Built Its Own Library System (1/2)

Duration: 10 minutes | No technical jargon — just the story!

- ① A small school has 500 students and a library with 5,000 books. The librarian, Mrs. Sharma, keeps everything in a thick register — handwritten entries for every book, every student, every borrow/return.
- ② One day, the principal asks: 'How many Science books do we have?' Mrs. Sharma spends 3 HOURS flipping through 200 pages, counting manually. The answer: 347.
- ③ The principal then asks: 'Which students have overdue books right now?' Another 2 hours of cross-referencing. Mrs. Sharma is exhausted.

MAIN STORY: The School That Built Its Own Library System (2/2)

- ④ A computer teacher offers to help. He says: 'First, I need a PLACE to store everything.' He creates a DATABASE — like buying a new empty filing cabinet.
- ⑤ Then he says: 'Inside this cabinet, I need SECTIONS.' He creates SCHEMAS — one section for 'books', one for 'students', one for 'borrowing records'. Like labeled drawers inside the cabinet.
- ⑥ Inside each section, he creates TABLES — like designing the columns on a new register page. The books table has columns for book_id, title, subject, author. The students table has student_id, name, class.
- ⑦ He types a few lines and suddenly, the answer to 'How many Science books?' appears in 0.01 seconds. Mrs. Sharma stares in disbelief.
- ⑧ 'That is SQL,' the teacher says. 'And those few lines I typed? CREATE DATABASE, CREATE SCHEMA, CREATE TABLE. That is exactly what we learn today.'

STORY → SQL CONNECTION

Every part of the library story maps to today's SQL concepts:

- ① Handwritten register = Unstructured data (no database)
- ② 3 hours to count books = No query engine → SQL answers in milliseconds
- ③ Cross-referencing records = Manual lookups → Databases link tables automatically
- ④ Filing cabinet = CREATE DATABASE — the top-level container
- ⑤ Labeled drawers = CREATE SCHEMA — logical sections inside the database
- ⑥ Register columns = CREATE TABLE — defining the structure with columns and types
- ⑦ Instant answer = SELECT query — the power of structured data
- ⑧ SQL commands = DDL (Data Definition Language) — building the structure



WhatsApp Announcement

Setup & Onboarding! Yesterday you learned WHAT databases are. Today you'll learn to BUILD them. By the end of today, you'll CREATE databases, schemas, and tables — the same skill that database administrators at Amazon and Flipkart use daily. You're not just learning SQL — you're building the backbone of every application. Let's create something! 🛠️

"Knowing SQL theory is like reading a recipe. Today, you COOK." — Siraj Sir

MINI STORY

Imagine you just bought a brand-new calculator. Before solving complex equations, you test it: $1 + 1 = ?$ If it says 2, you trust it.

SQL works the same way! Before building tables and querying millions of rows, you can use `SELECT` as a calculator, a clock, and a system inspector — all without needing a single table.

DEFINITION



DEFINITION: SQL Query



Technical:

An SQL query is a structured instruction written in Structured Query Language that communicates with a relational database management system (RDBMS) to perform operations such as retrieving, inserting, updating, or deleting data. The most basic form is SELECT expression;



Layman's:

Think of SQL like ordering food at a restaurant. You tell the waiter (database) exactly what you want: 'Give me today's date' or 'Calculate 100 times 18'. The waiter brings back exactly what you asked for. That request is a 'query'.

SYNTAX: Basic Queries

-- Basic structure

```
SELECT 42;
```

-- As a calculator

```
SELECT 1 + 1;
```

-- Addition

```
SELECT 100 * 0.18;
```

-- Multiplication

```
SELECT 2025 - 1986;
```

-- Subtraction

-- System info (no FROM needed)

```
SELECT version();
```

-- PostgreSQL version

```
SELECT current_date;
```

-- Today's date

```
SELECT current_time;
```

-- Current time

```
SELECT current_timestamp;
```

-- Date + Time

```
SELECT current_user;
```

-- Who's logged in

```
SELECT current_database();
```

-- Which database

-- String operations

```
SELECT 'Hello' || ' ' || 'World!';
```

-- Concatenation

```
SELECT UPPER('hello');
```

-- HELLO

```
SELECT LENGTH('PostgreSQL');
```

-- 10

PROBLEM: You just installed PostgreSQL. You want to verify the installation is working before doing anything else.

🤔 Think about it: What's the simplest way to test if the database engine responds?

💡 What happens: PostgreSQL returns a long string showing the exact version, compiler, and platform. If you see this, your installation is healthy!

Solution

✓ ANSWER

```
SELECT version();
```

PROBLEM: You want to check today's date, the current time, and which database you're connected to — all in one query.

🤔 Think about it: Can you combine multiple SELECTs into one?

💡 What happens: Returns one row with 4 columns. AS gives each column a friendly name (alias). This is your first multi-column SELECT!

Solution



ANSWER

```
SELECT
  current_date AS today,
  current_time AS right_now,
  current_database() AS connected_to,
  current_user AS logged_in_as;
```

PROBLEM: You need a comprehensive system status report combining version, user, database, and timestamp in a single query.

🤔 Think about it: Can you combine system functions with calculations?

💡 What happens: Returns database name, user, full version string, exact timestamp, and your session's process ID. DBAs use this to identify sessions.

Solution



```
SELECT
  current_database() AS database_name,
  current_user AS logged_in_user,
  version() AS postgres_version,
  current_timestamp AS report_generated_at,
  pg_backend_pid() AS session_process_id;
```

PROBLEM: Use SQL as a calculator to compute GST. A product costs ₹999 before tax. GST is 18%. What's the total?

🤔 Think about it: SQL can do math without any table!

💡 What happens: Returns 999, 179.82, 1178.82, 1178.82. ROUND() controls decimal places. Every billing system does this calculation!

Solution

✓ ANSWER

```
SELECT
  999 AS base_price,
  999 * 0.18 AS gst_amount,
  999 * 1.18 AS total_with_gst,
  ROUND(999 * 1.18, 2) AS rounded_total;
```

KEY TAKEAWAYS

SELECT is the most versatile SQL command. It works as a calculator, a clock, a system inspector, and (starting tomorrow) a data retriever. You don't always need a table — `SELECT expression;` works on its own.

PRO TIP

PRO TIP

Always end every SQL statement with a semicolon (;). PostgreSQL waits patiently for the semicolon before executing. Forgetting it is the #1 beginner mistake — your query appears to hang, but it's just waiting for you to finish!

COMMON MISTAKE: Forgetting the Semicolon

The Mistake:

Typing `SELECT 1 + 1` and pressing Execute. Nothing happens. The cursor just moves to the next line.

The Fix:

Every SQL statement **MUST** end with a semicolon: `SELECT 1 + 1;` — the semicolon tells PostgreSQL 'I'm done, execute this now!'

INTERVIEW SPOTLIGHT

Q: 'Can you run a SELECT statement without a FROM clause?'

A: 'Yes! In PostgreSQL, SELECT can be used as a calculator (SELECT 1+1;), to call system functions (SELECT version();), or to get current date/time (SELECT NOW();). No table is needed for these expressions.'

CHALLENGE

CHALLENGE

TRY THIS

- 1.: Run `SELECT version();` and note which PostgreSQL version you have.
- 2.: Calculate your age in days: `SELECT current_date - '2000-01-15';` (use your birthday)



THINK ABOUT IT: The AS Keyword

You saw ``AS today`` in the examples. That's called a column alias. It doesn't change the stored data — it just renames the column in the OUTPUT.

Without alias: ``SELECT current_date;`` → column header appears as `'current_date'`.

With alias: ``SELECT current_date AS today;`` → column header appears as `'today'`.

The AS keyword is technically OPTIONAL: ``SELECT current_date today;`` also works. But ALWAYS use AS explicitly — it makes your SQL 10× more readable and it's an interview expectation. Never skip AS in production code.

INTERVIEW SPOTLIGHT

Q: 'What is the difference between `current_timestamp` and `NOW()`?'

A: 'Functionally they are identical in PostgreSQL — both return the current date and time WITH timezone.

`current_timestamp` is defined in the ANSI SQL standard (works on all databases). `NOW()` is PostgreSQL-specific and a little shorter to type. In portable code that may run on MySQL or Oracle, prefer `current_timestamp`. In PostgreSQL-only code, either is fine.'

PRO TIP

 Pro Tip: SELECT Without a Table is a Secret Superpower

Most tutorials teach you that every SELECT needs a FROM clause. That's half-true — in PostgreSQL, you can ``SELECT 1 + 1;`` with no FROM and it works. Why? Because PostgreSQL has an invisible pseudo-table called the 'dual' concept under the hood.

Why is this useful? Because it turns SQL into a calculator that you can use for:

- Quick arithmetic when you don't have a Python interpreter open
- Testing functions BEFORE running them on real data (``SELECT UPPER('test');``)
- Checking system state (``SELECT version();``, ``SELECT current_user;``)
- Generating dummy rows for testing (``SELECT generate_series(1, 10);`` returns 10 rows!)

Every senior SQL engineer uses this daily. Now you can too.

DEFINITION

You learned to build tables by hand — now load the WHOLE dataset at once

You could create all 16 schemas and 55 tables manually... or run ONE script that creates the database, builds every schema and table, and imports all ~2.7 million rows of real data. That script ships inside the course repo.

Step 1 — Get the COMPLETE repo (once)

```
# Option A: clone with git
git clone https://github.com/starlordali4444/postgresql.git
cd postgresql

# Option B: on the repo page, click the green 'Code' button ->
#             'Download ZIP', unzip it, then cd into the folder.
```

Step 2 — Load everything (run from the repo ROOT)

```
psql -U postgres -f datasets/v3/sql/setup_accio_retailmart.sql
```

```
-- Creates a database named accio_retailmart_ (trailing underscore!),  
-- builds 16 schemas + 55 tables, and \copy-loads every CSV.  
-- Takes ~1-2 minutes. Run from the repo ROOT so the  
-- 'datasets/v3/csv/...' paths inside the script resolve.
```

Step 3 — Rename it to YOUR batch (in pgAdmin 4)

The script creates the database as `accio_retailmart_` — with a dangling underscore on purpose. Make it yours:

- In pgAdmin 4, expand 'Databases' in the left tree.
- Right-click `accio_retailmart_` -> Rename...
- Add your batch number, e.g. `accio_retailmart_NN`, and save.

Renaming a database is a real DBA task — good first hands-on practice in pgAdmin.

Step 4 — Verify it worked

```
\c accio_retailmart_NN -- use YOUR batch number here
SELECT count(*) FROM sales.orders; -- expect 150,000
```


Why download the COMPLETE repo?

The setup script loads CSVs using paths relative to the repo root (datasets/v3/csv/...). With the whole repo, those paths just work — and you keep the practice questions, notes and curriculum together in one place. Always run the command from the repo ROOT, never from inside a subfolder.

The database arrives named `accio_retailmart_` on purpose — renaming it to your batch number (e.g. `accio_retailmart_NN`) in pgAdmin 4 is your first hands-on database task.

Q1 — What are the 5 categories of SQL commands?

DDL (Data Definition): CREATE, ALTER, DROP, TRUNCATE — defines structure.

DML (Data Manipulation): INSERT, UPDATE, DELETE — changes rows.

DQL (Data Query): SELECT — reads data. This is 90% of an analyst's work.

DCL (Data Control): GRANT, REVOKE — permissions.

TCL (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT.

Today we live in DQL. Next session we start on DDL.

Q2 — What is the difference between a database and a schema?

A database is the whole building; a schema is a department inside it. One PostgreSQL database holds many schemas, and each schema holds tables. RetailMart has 16 schemas (sales, customers, hr, finance...) inside ONE database. You connect to a DATABASE, then query TABLES that live inside SCHEMAS.

Q3 — What is a schema-qualified table name and why does it matter?

schema.table — for example sales.orders. Without the schema prefix PostgreSQL looks only in your search_path (usually public), so an unqualified 'orders' will fail even though sales.orders exists.

On a 16-schema database like RetailMart, always qualify. It is the single most common beginner error.

Q4 — How do you explore an unfamiliar database?

In psql: \l lists databases, \c connects to one, \dn lists schemas, \dt lists tables, \d tablename shows a table's columns and types.

That is the exact sequence a new joiner runs on day one at any company — before writing a single query.

Q5 — Why always use LIMIT when exploring a new table?

Because you have no idea how big it is. `SELECT * FROM web_events.page_views` on a production table can return tens of millions of rows, freeze your client and load the server.

`SELECT * FROM sales.orders LIMIT 10;` shows you the shape of the data instantly and safely. Explore with LIMIT, aggregate later.

Q6 — What does **SELECT** without a **FROM** clause do?

It evaluates expressions — PostgreSQL treats it as a calculator: `SELECT 2+2`; `SELECT now()`; `SELECT version()`;

Useful for testing functions and checking your server: `SELECT current_database(), current_user, version()`;

8 Rapid-Fire Questions — 90 Seconds Total

Close this notebook. Take a blank paper. Answer all 8 in 90 seconds. Score yourself — 6+ = confident, 4-5 = review the session, under 4 = redo the whole day.

- 1.: Which psql command lists all schemas in the current database?
- 2.: Write a query that returns the first 5 rows of sales.orders.
- 3.: Why does `SELECT * FROM orders;` fail when sales.orders exists?
- 4.: Which SQL category does `SELECT` belong to?
- 5.: How do you see the columns and types of customers.customers?
- 6.: What does `SELECT current_database();` tell you?
- 7.: How many schemas and tables does RetailMart V3 have?
- 8.: Why is `LIMIT` important when exploring an unknown table?

ANSWER KEY — Check Yourself

1. \dn
2. `SELECT * FROM sales.orders LIMIT 5;`
3. The table is in the sales schema, not in your search_path — qualify it as sales.orders.
4. DQL (Data Query Language) — the SELECT family.
5. \d customers.customers
6. The name of the database your session is currently connected to.
7. 16 schemas, 55 tables, about 2.7 million rows.
8. Without it a single SELECT can return millions of rows and hang your client.

PRO TIP

Pass 1 — Read + Code Along (45 min)

Open your PostgreSQL terminal. Read each section, then actually TYPE and run every psql command and query shown. Don't copy-paste — muscle memory matters.

Pass 2 — Recall without Notes (20 min)

Close the notebook. Try to write these from memory: (a) the psql command that lists schemas, (b) the one that shows a table's columns, (c) a LIMIT query on sales.orders, (d) the three system-info functions. Open the notes only when you're stuck — that struggle is where learning happens.

Pass 3 — Teach It (15 min)

Explain to an imaginary junior (or a friend): 'What is the difference between a database, a schema and a table — and why must I write sales.orders instead of orders?' If you can teach it cleanly, you own it. If you stumble — you've found the weak spot. Re-read that section.

 Total time: 80 minutes. Do this and you will SAIL through earlier sessions.

CHALLENGE

CHALLENGE (1/2)

THE 6-STEP CHALLENGE

You have 30 minutes. No Googling syntax. Only today's notes. Everything below is READ-ONLY — you are exploring, not changing anything.

CHALLENGE (2/2)

Step 1 —: Connect to your RetailMart database and list every schema. How many are there?

Step 2 —: List the tables inside the sales schema. Write down how many you find.

Step 3 —: Show the structure of sales.orders. Note every column name and its type — you will need these all course long.

Step 4 —: Return the first 10 rows of sales.orders. Then the first 10 of customers.customers.

Step 5 —: Run `SELECT current_database(), current_user, version();` and save the output.

Step 6 —: Find 3 columns whose names surprised you (not what you would have guessed) and write them down.

 **Bonus:** Keep a notes file of the tables and columns you meet. The analysts who move fastest are the ones who learned the schema early — not the ones who memorised syntax.

REAL WORLD



THE FIRST WEEK AT ANY DATA JOB

Nobody hands you a diagram. You get a connection string and a question. What you did today IS the job's first week:

- Connect to a database you have never seen before
- List the schemas and tables to learn the landscape
- Inspect a few tables to learn the real column names
- Run small LIMIT queries before anything expensive
- Ask what the data MEANS — the part no tool can do for you

Analysts who skip this step write confident queries against columns that do not exist. You just avoided that.

DEFINITION

You can now explore any database. Next you start BUILDING one.

- Today: connected, explored 16 schemas, ran your first real queries
- Next session: DDL — CREATE DATABASE, CREATE SCHEMA, CREATE TABLE, DROP
- You will build every column as TEXT first — on purpose
- Then data types arrive to fix the pain that all-TEXT causes

Onboarding & First Queries — Quick Reference

-- Explore (psql)

```
\l          -- list databases
\c accio_retailmart_NN -- connect
\dn         -- list schemas
\dt sales.* -- list tables in a schema
\d sales.orders -- columns + types
```

-- First Queries

```
SELECT * FROM sales.orders LIMIT 10;
SELECT order_id, order_date, net_total
FROM sales.orders
LIMIT 5;
```

-- System Info

```
SELECT version();
SELECT current_database();
SELECT current_user;
SELECT now();
```